Sistemas Distribuídos

DHT Chord

Vladimir Rocha (Vladi)

CMCC - Universidade Federal do ABC

Disclaimer

- Estes slides foram baseados no tutorial do professor Amir Paybereah https://payberah.github.io/files/download/p2p/chord.pdf

Tabela de Hash Distribuída

- Denominada DHT (Distributed Hash Table)
- Sistema P2P estruturado (Peer atua como cliente e servidor)
- Criada em 2001 para encontrar um recurso de forma eficiente: O(log N)
- Provê escalabilidade de tamanho e particionamento (sharding).
- Evita o ponto único de falha (SPoF)
- Usada em:



Conceito

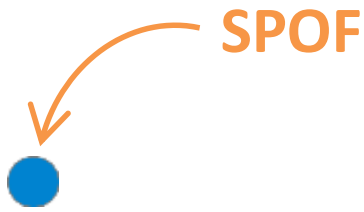
- Considere uma tabela de hash normal.

Key	Value
Fatemeh	Stockholm
Sarunas	Lausanne
Tallat	Islamabad
Cosmin	Bucharest
Seif	Stockholm
Amir	Tehran

Conceito

- Considere uma tabela de hash normal. Se ela estiver só em um servidor, esse se transforma no chamado "ponto único de falha" (SPOF – Single Point of Failure) caso morra.

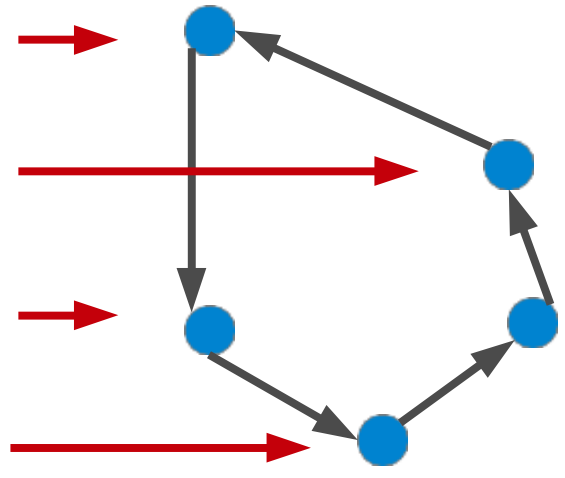
Key	Value
Fatemeh	Stockholm
Sarunas	Lausanne
Tallat	Islamabad
Cosmin	Bucharest
Seif	Stockholm
Amir	Tehran



Conceito

- Então, que tal distribuir a tabela de hash em vários nós?
  Ou seja, teremos uma tabela de hash **distribuída**

Key	Value
Fatemeh	Stockholm
Sarunas	Lausanne
Tallat	Islamabad
Cosmin	Bucharest
Seif	Stockholm
Amir	Tehran



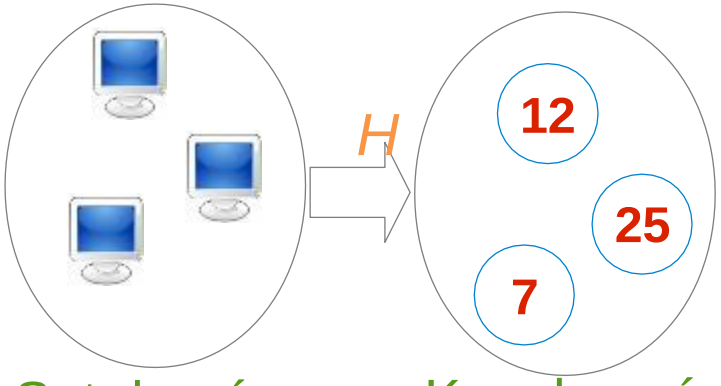
Chord (DHT estruturado em anel)

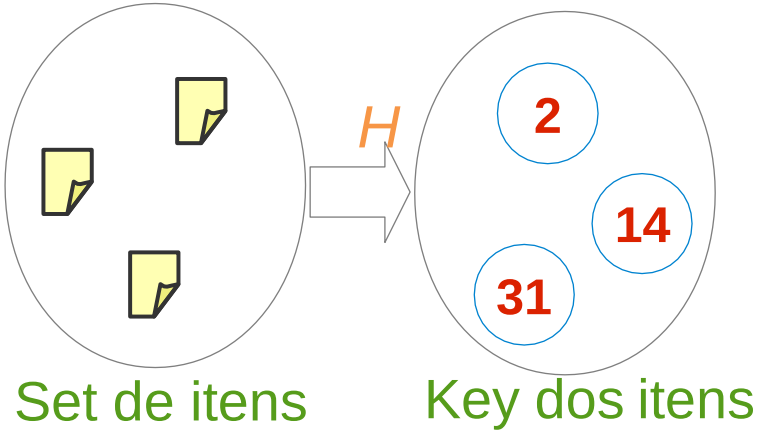
- Visão geral da construção e utilização

Chord (DHT estruturado em anel)

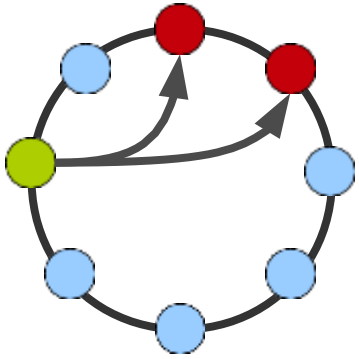
- Visão geral da construção e utilização

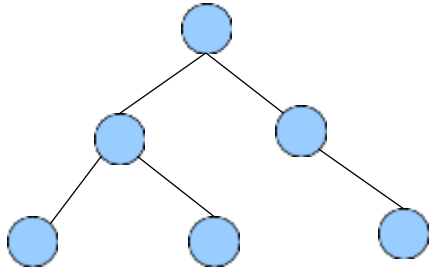
1. Definir o espaço comum de valores para os nós e itens



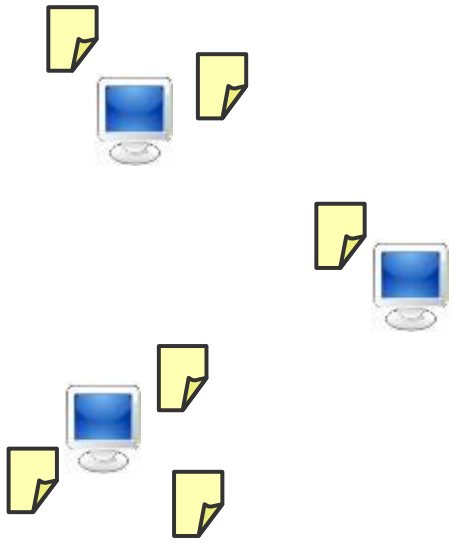


2. Conectar os nós usando um número pequeno de links (minimizar hops)





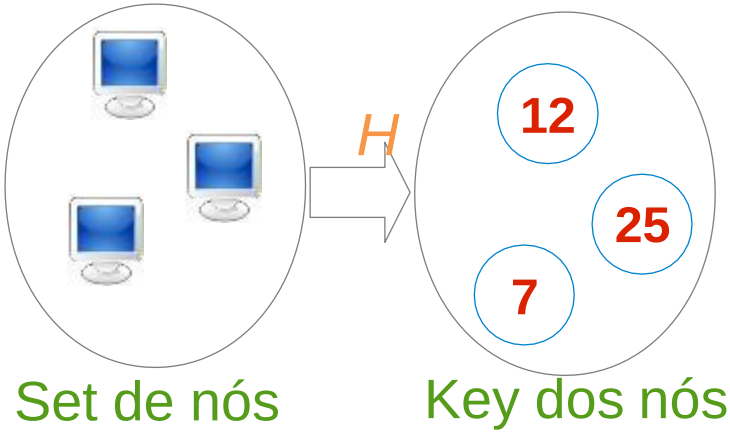
3. Definir a estratégia para designar itens aos nós

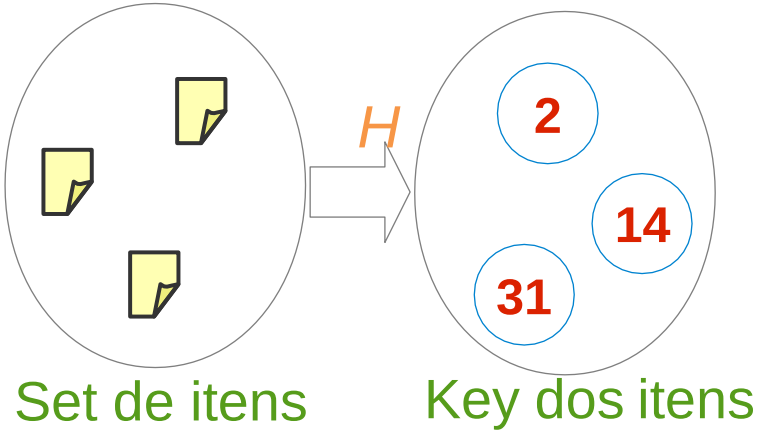


Chord (DHT estruturado em anel)

- Visão geral da construção e utilização

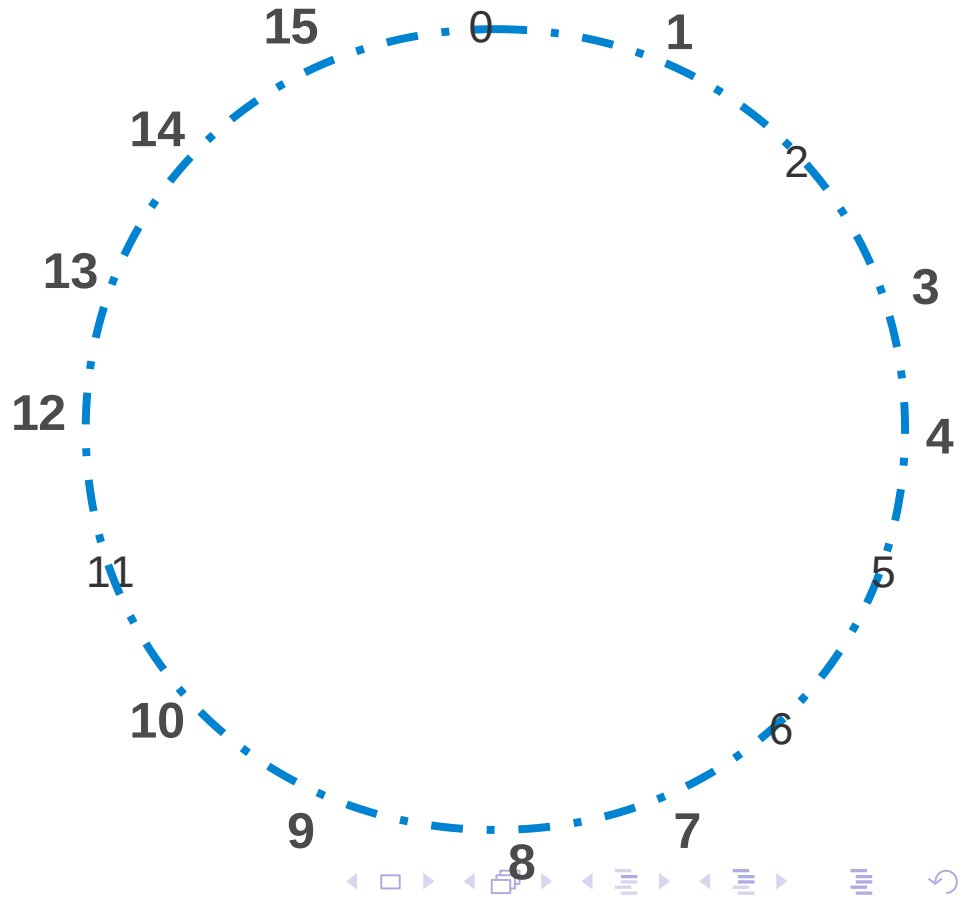
1. Definir o espaço comum de valores para os nós e itens





Construção da DHT Chord

- Usar um espaço de nomes (espaço de identificadores) de {0, 1, 2 ... 2^M-1}
  - Esse espaço de nomes formará um anel lógico módulo 2^M



Construção da DHT Chord

- Usar um espaço de nomes (espaço de identificadores) de {0, 1, 2 ... 2^M-1}
  - Esse espaço de nomes formará um anel lógico módulo 2^M
- O nó terá um nº aleatório obtido de uma função de **hash H**

Exemplo:

Espaço 2^M = 16 {0, ..., 15}

Cinco nós a, b, c, d, e

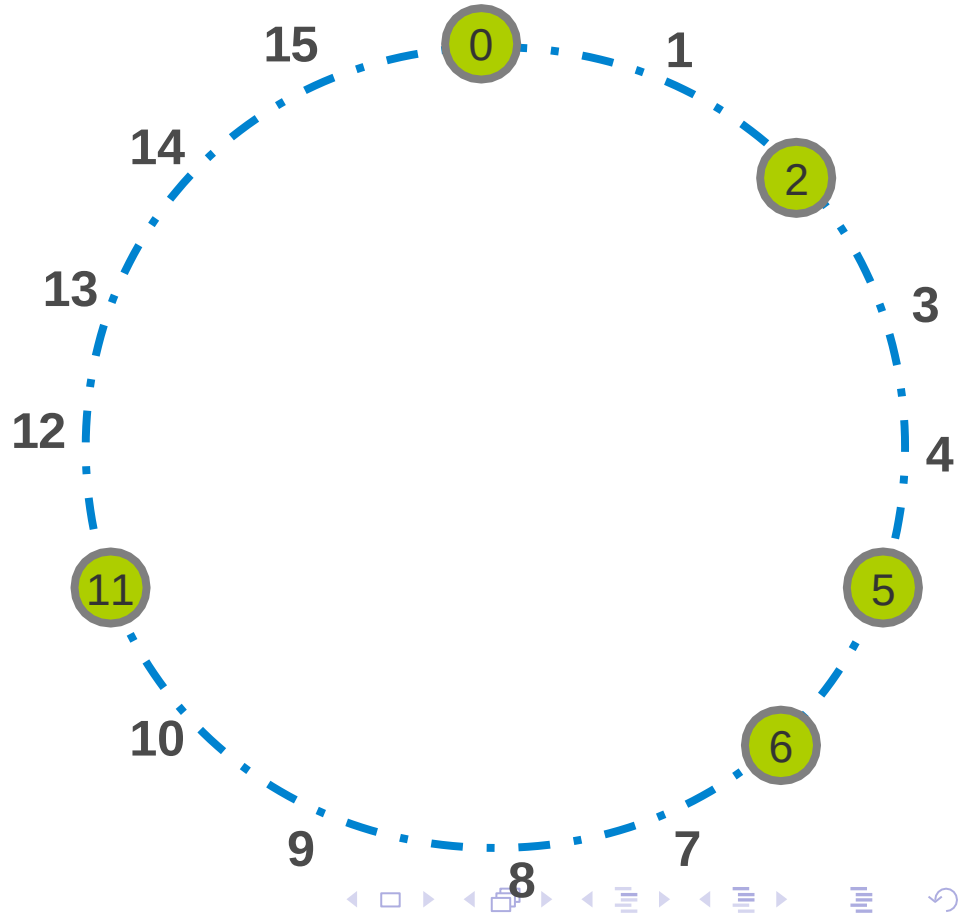
H(a) = 6

H(b) = 5

H(c) = 0

H(d) = 11

H(e) = 2



Construção da DHT Chord

- Usar um espaço de nomes (espaço de identificadores) de {0, 1, 2 ... 2^M-1}
  - Esse espaço de nomes formará um anel lógico módulo 2^M
- O nó terá um nº aleatório obtido de uma função de **hash H**
- O nó será responsável por um conjunto disjunto de identificadores

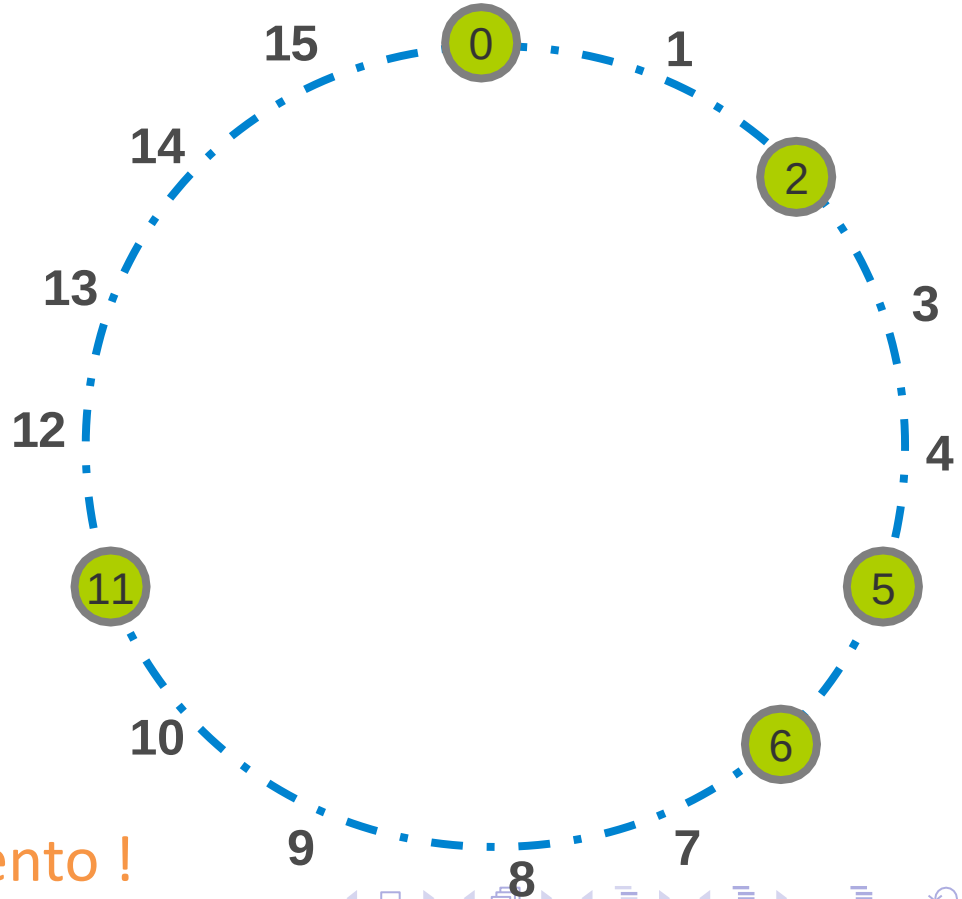
  [nº nó anterior+1, nº dele]

Exemplos:

Nó 0: [12, 0]

Nó 5: [3, 5]

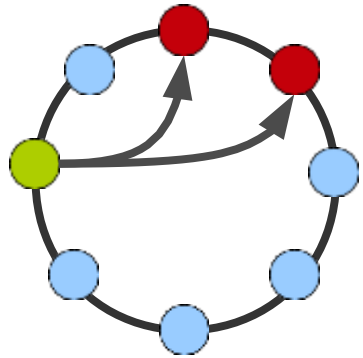
Nó 11: [7,11]



Conjunto disjunto ... Particionamento !

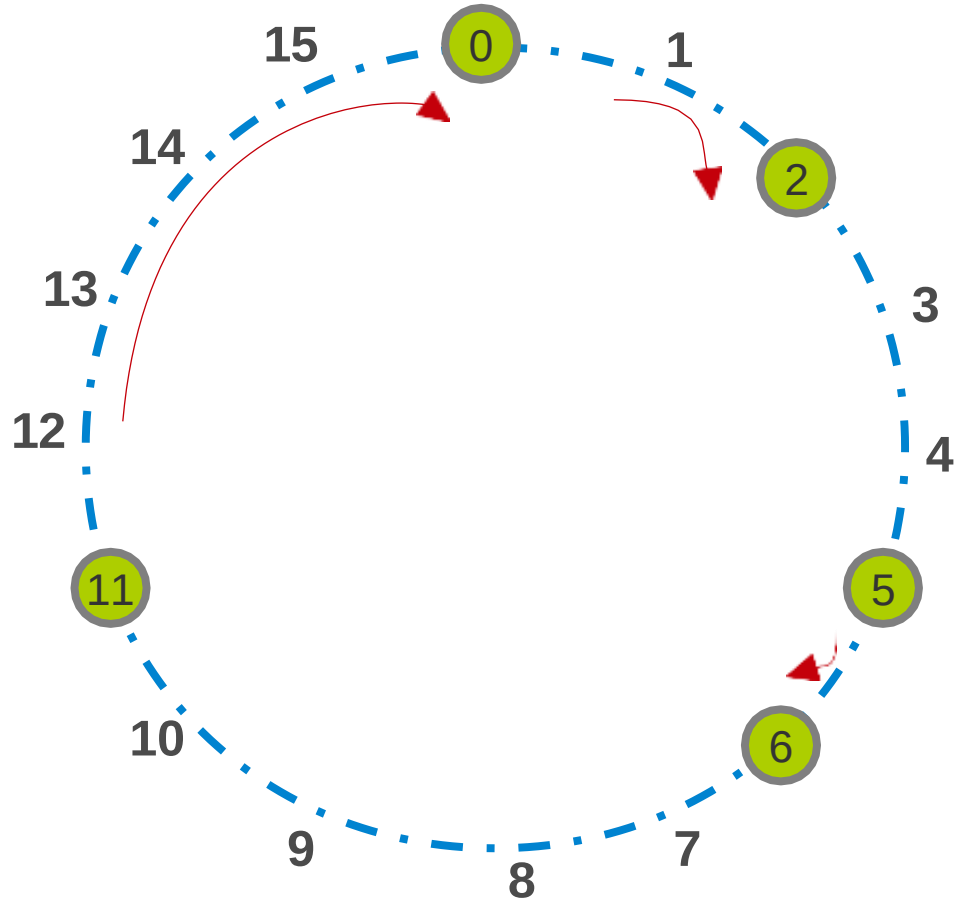
Chord (DHT estruturado em anel)

2. Conectar os nós usando um número pequeno de links (minimizar hops)



Construção da DHT Chord

- O sucessor de um identificador é o primeiro nó indo na direção do relógio começando no identificador



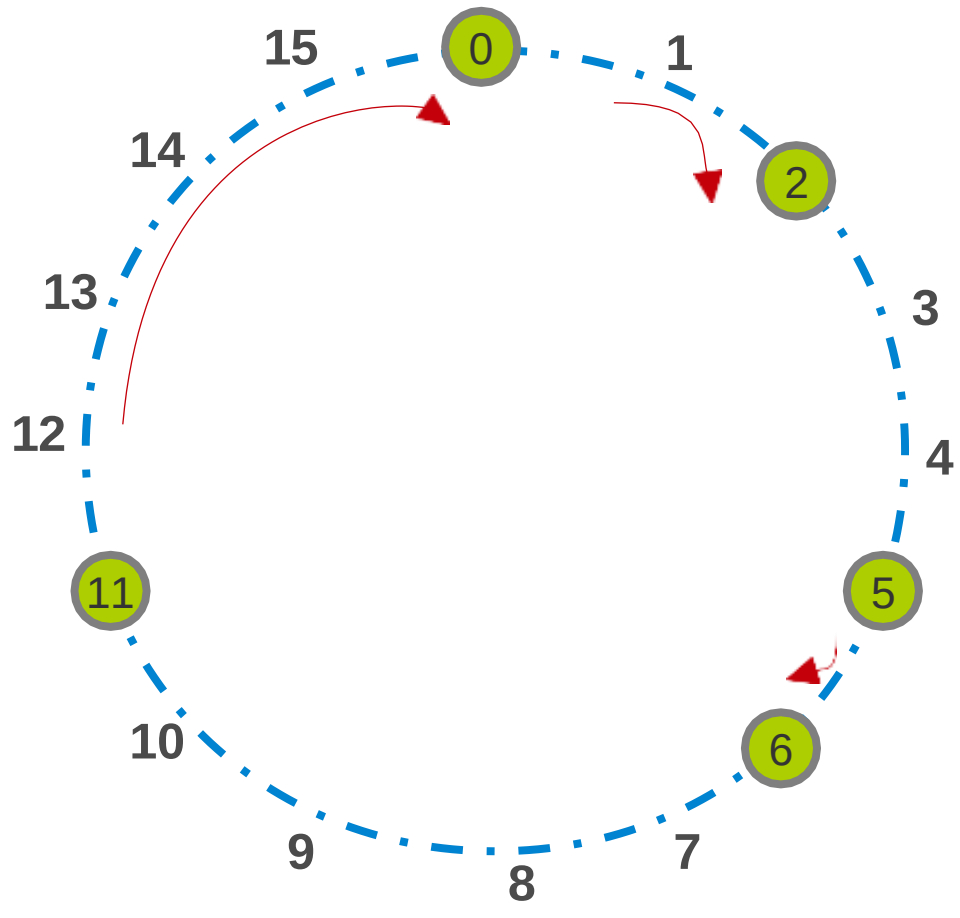
Construção da DHT Chord

- O sucessor de um identificador é o primeiro nó indo na direção do relógio começando no identificador

- `succ(x)` é o primeiro nó no anel com id **maior ou igual** a x

Exemplo:

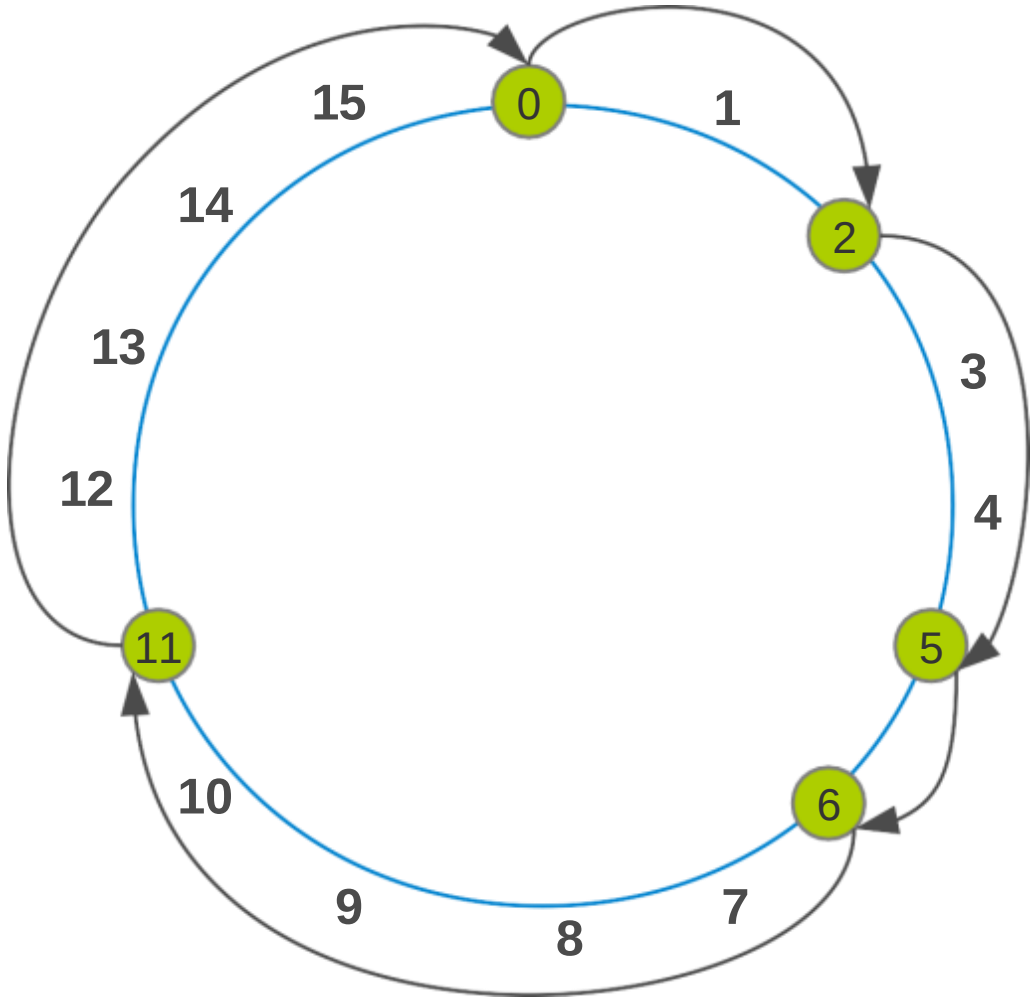
```
succ(12) = 0
succ(1)  = 2
succ(6)  = 6
```



Construção da DHT Chord

- Apontar cada nó *p* ao seu sucessor succ(*p*+1).

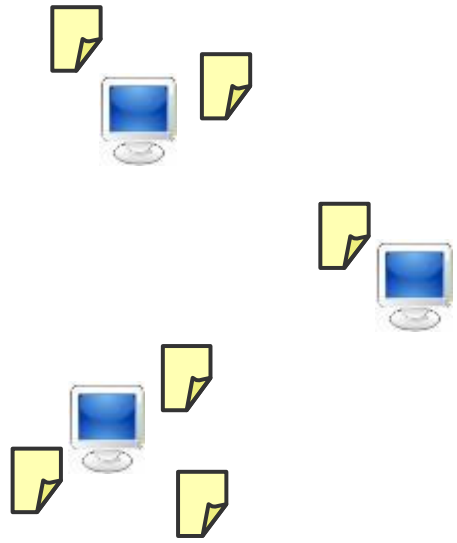
Sucessor do nó 0 é succ(1) = 2
Sucessor do nó 2 é succ(3) = 5
Sucessor do nó 5 é succ(6) = 6
Sucessor do nó 6 é succ(7) = 11
Sucessor do nó 11 é succ(12) = 0



Invariante: um nó sempre deverá ter a conexão com seu sucessor!

Chord (DHT estruturado em anel)

3. Definir a estratégia para designar itens aos nós



Estratégia para armazenar itens

- Onde armazenar os dados?
  - Utilizando uma função de hash H
  - A mesma da construção

- Cada item <key,value> obterá o identificador H(key) = k

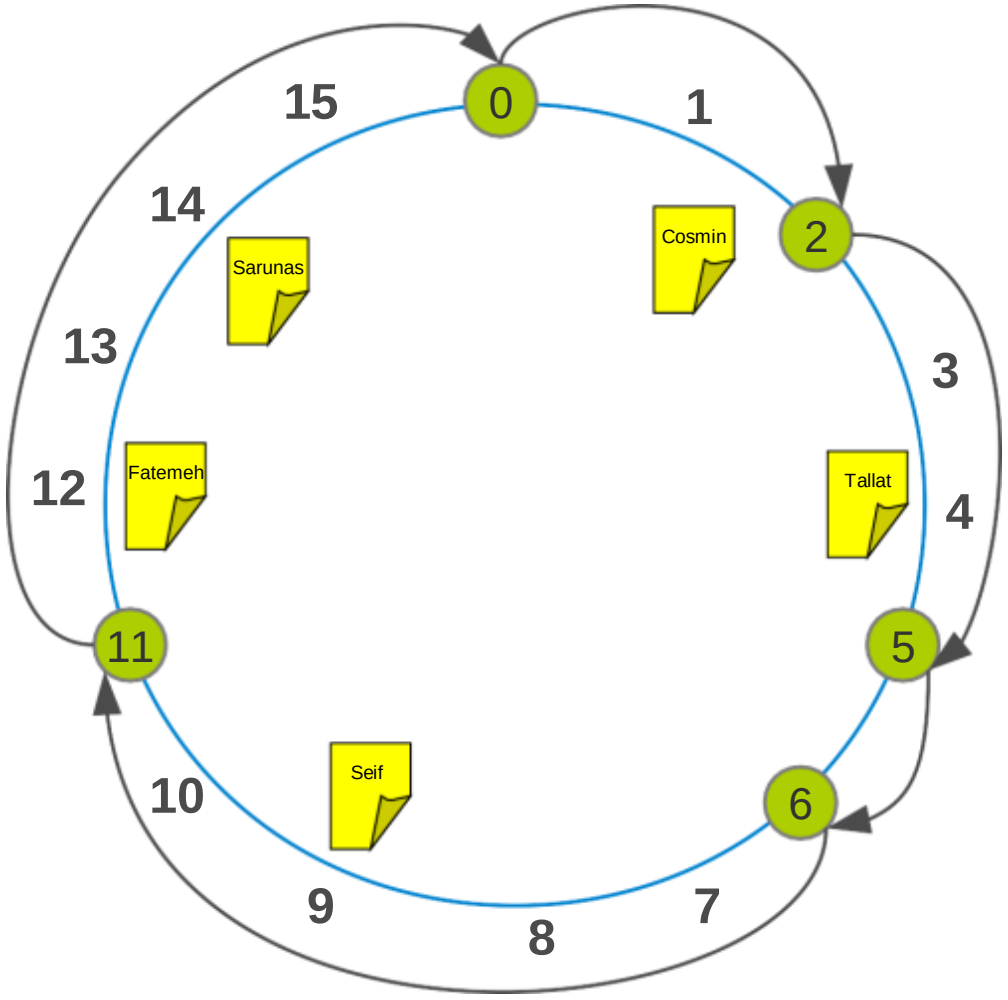
```
H('Fatemeh') = 12
H('Cosmin')  = 2
H('Seif')    = 9
H('Sarunas') = 14
H('Tallat')  = 4
```

Estratégia para armazenar itens

- Onde armazenar os dados?
  - Utilizando uma função de hash H
  - A mesma da construção

- Cada item <key,value> obterá o identificador H(key) = k

```
H('Fatemeh') = 12
H('Cosmin')  = 2
H('Seif')    = 9
H('Sarunas') = 14
H('Tallat')  = 4
```



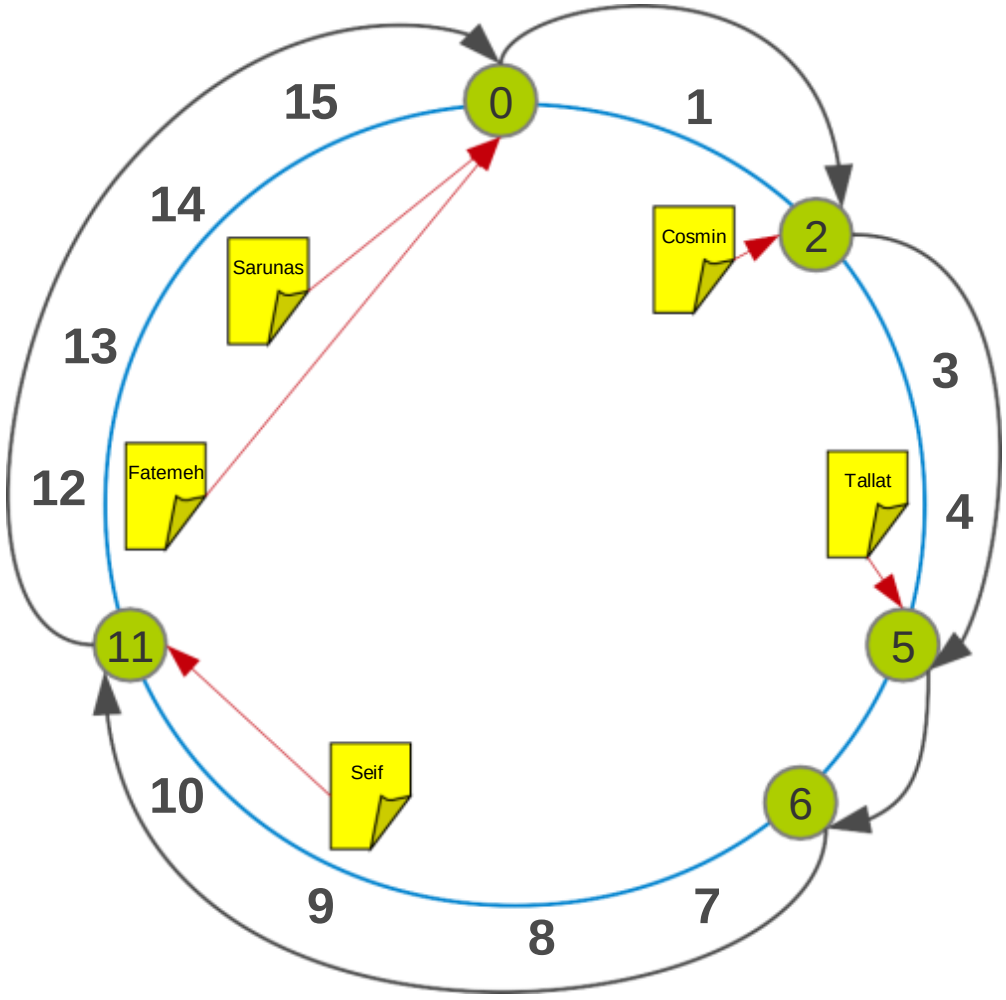
Estratégia para armazenar itens

- Onde armazenar os dados?
  - Utilizando uma função de hash H
  - A mesma da construção

- Cada item <key,value> obterá o identificador H(key) = k

```
H('Fatemeh') = 12
H('Cosmin')  = 2
H('Seif')    = 9
H('Sarunas') = 14
H('Tallat')  = 4
```

- Armazene cada item no sucessor



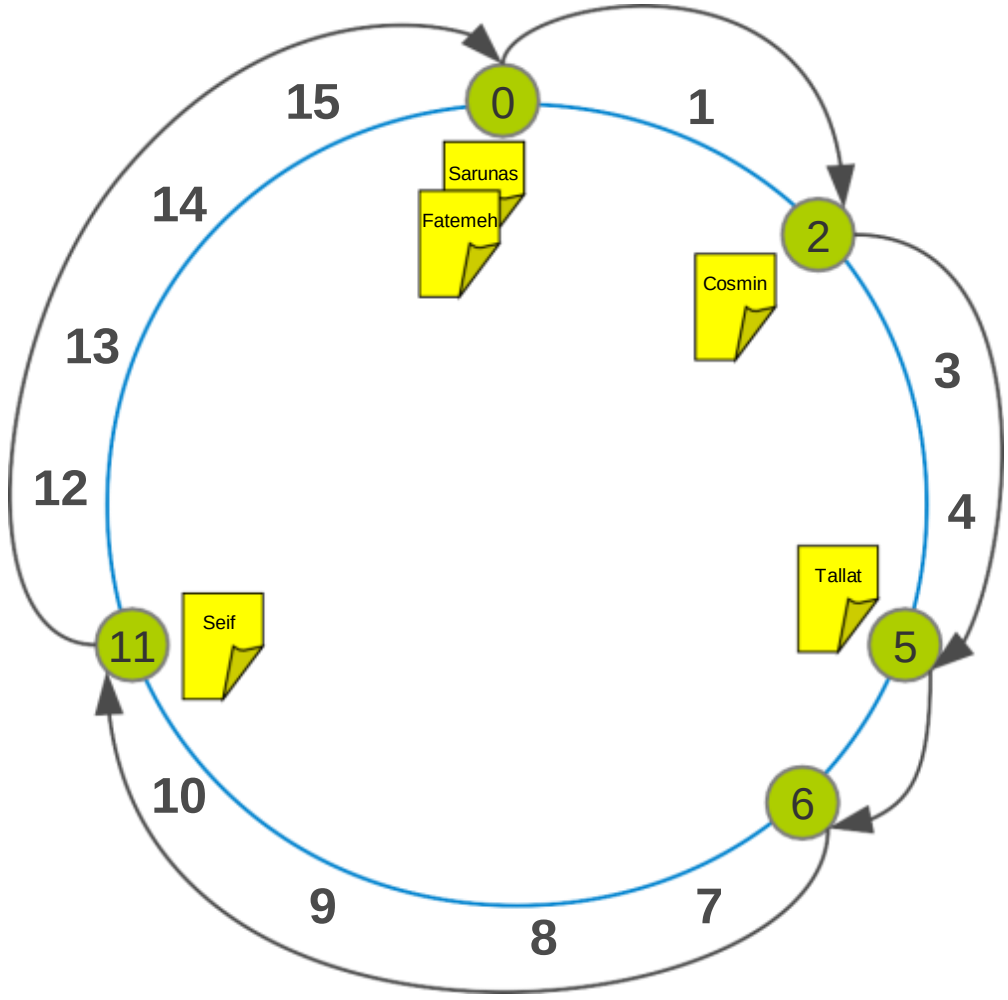
Estratégia para armazenar itens

- Onde armazenar os dados?
  - Utilizando uma função de hash H
  - A mesma da construção

- Cada item <key,value> obterá o identificador H(key) = k

```
H('Fatemeh') = 12
H('Cosmin')  = 2
H('Seif')    = 9
H('Sarunas') = 14
H('Tallat')  = 4
```

- Armazene cada item no sucessor



Estratégia para armazenar itens: *consistent hashing*

- Chord utiliza *consistent hashing* para designar chaves aos nós

- ***Consistent Hashing*** : técnica de hashing que permite que somente O(m/N) pares de chave-valor sejam transferidas na adição ou remoção de nós

  m é o número de chaves armazenadas.
  N é o número de nós.

Para pensar ...

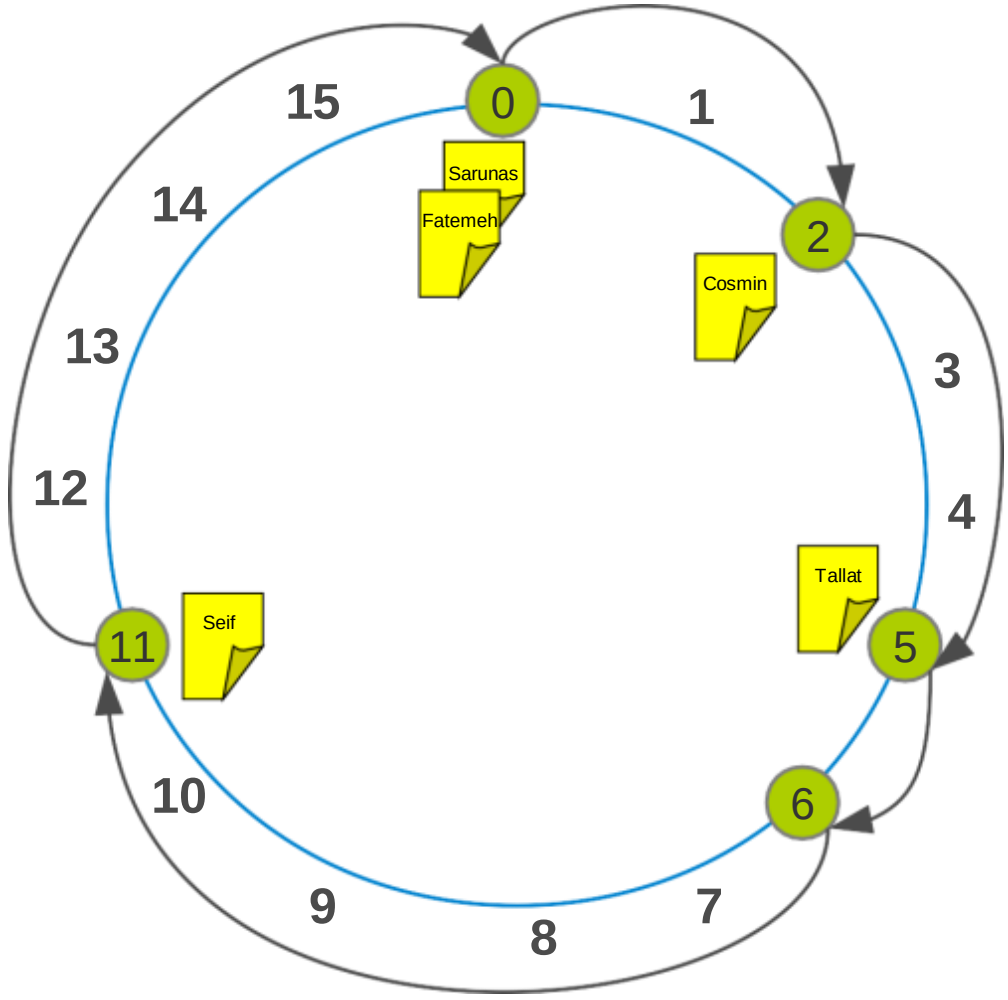
Em vez de armazenar o item no sucessor (como mostrado no slide anterior) qual seria o problema de designar a chave ao nó utilizando a função módulo e não usando o consistent hashing?

Ou seja, qual o problema em armazenar o item no nó: H(key) % N

Como realizar uma busca?

Como realizar uma busca?

- Para buscar uma chave k
  - Pergunte para **qualquer** nó do anel
  - Calcule H(k)
  - Siga os apontadores dos succ até k ser encontrado



Como realizar uma busca?

- Para buscar uma chave k
  - Pergunte para **qualquer** nó do anel
  - Calcule H(k)
  - Siga os apontadores dos succ até k ser encontrado

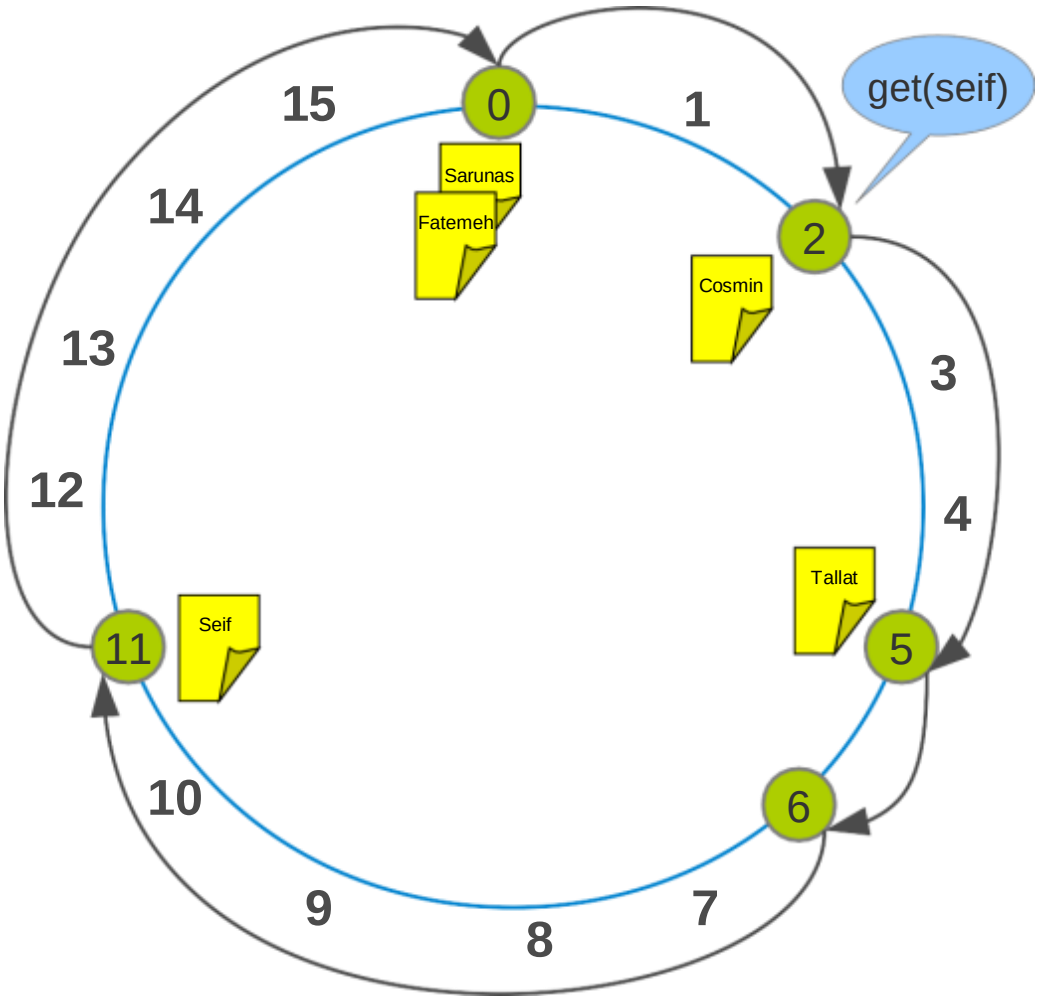
Exemplo, procurando "Seif" em nó 2

```
H('Seif') = 9
```

Percorrer nós 2, 5, 6, 11

Devolver "Stockholm" ao iniciador

Key	Value
Seif	Stockholm



Algoritmos

```
// Estrutura do nó n
No {
    int meuId = n
    No predecessor = null
    No sucessor = null
}
```

Algoritmos

- (a, b] é o segmento do anel que não inclui a mas inclui b
- n.foo(...) é uma chamada RPC ao nó n pelo método foo
- n.bar é uma chamada RPC ao nó n obtendo a variável bar

```
// pergunta ao nó n encontrar o successor do id procurado
procedure n.findSuccessor(id, No ini) {
    if (predecessor ≠ nil and id ∈ (predecessor, n]) then ini.response(id, n)
    else if (id ∈(n, successor]) then ini.response(id, sucessor)
    else // encaminha a pergunta pelo anel
        sucessor.findSucessor(id,ini)
}
```

Algoritmos

```
procedure n.put(id, value) {


}
procedure n.get(id) {


}
```

Algoritmos

```
procedure n.put(id, value) {
   op = put
   findSuccessor(id, this)
}

procedure n.get(id) {
   op = get
   findSuccessor(id, this)
}

procedure n.response(id, node) {
   if (op == put) node.store(id,value)
   else if (op == get) return node.retrieve(id)
}
```

- Ambos métodos put e get utilizam a chamada a findSucessor

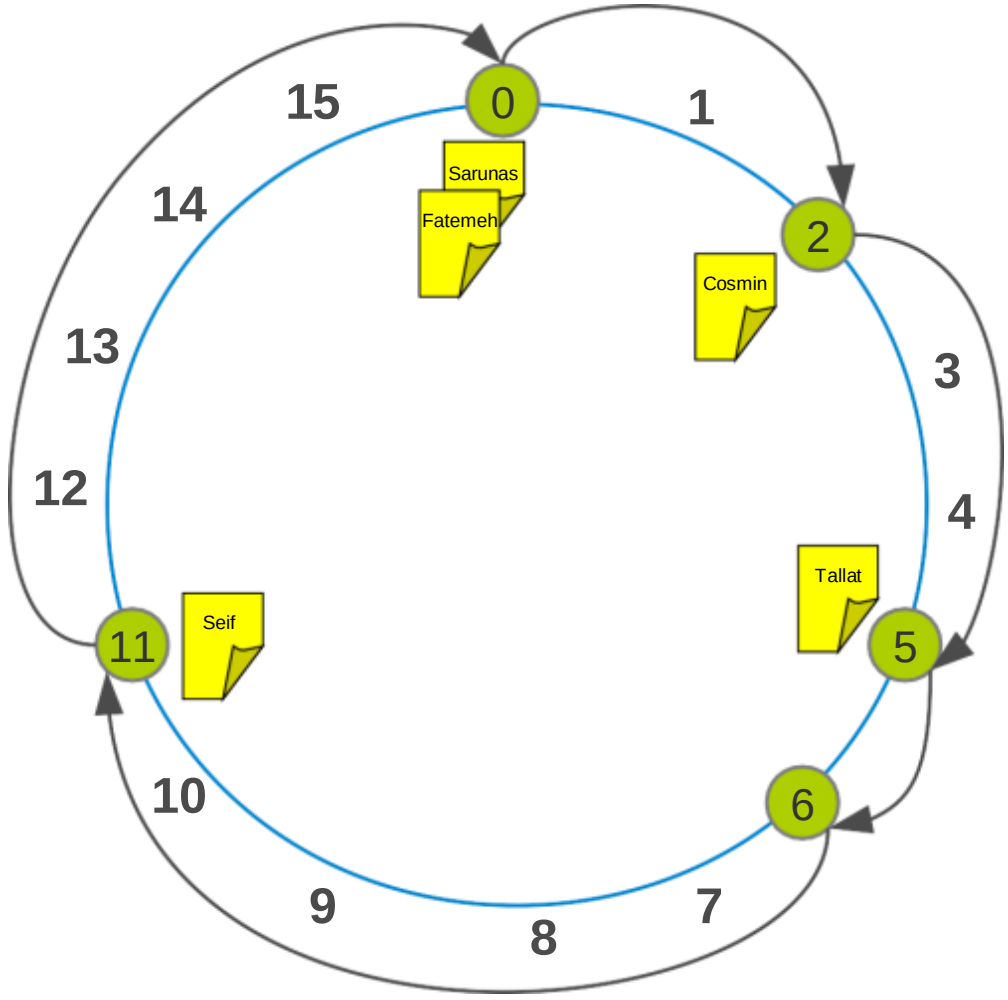
Algoritmos

```
// Estrutura do nó n (inicial)
No {
    int meuId = n
    No predecessor = null
    No sucessor = null
}

// Modificando a estrutura do nó n (agora considerando store/retrieve)
No {
    int meuId = n
    No predecessor = null
    No sucessor = null
    KV chaves-valor = null
}
```

Buscas mais rápidas

- Se somente o apontador ao sucessor for usado: O(N) para N nós
- Melhor usar "atalhos" (*shortcuts*)

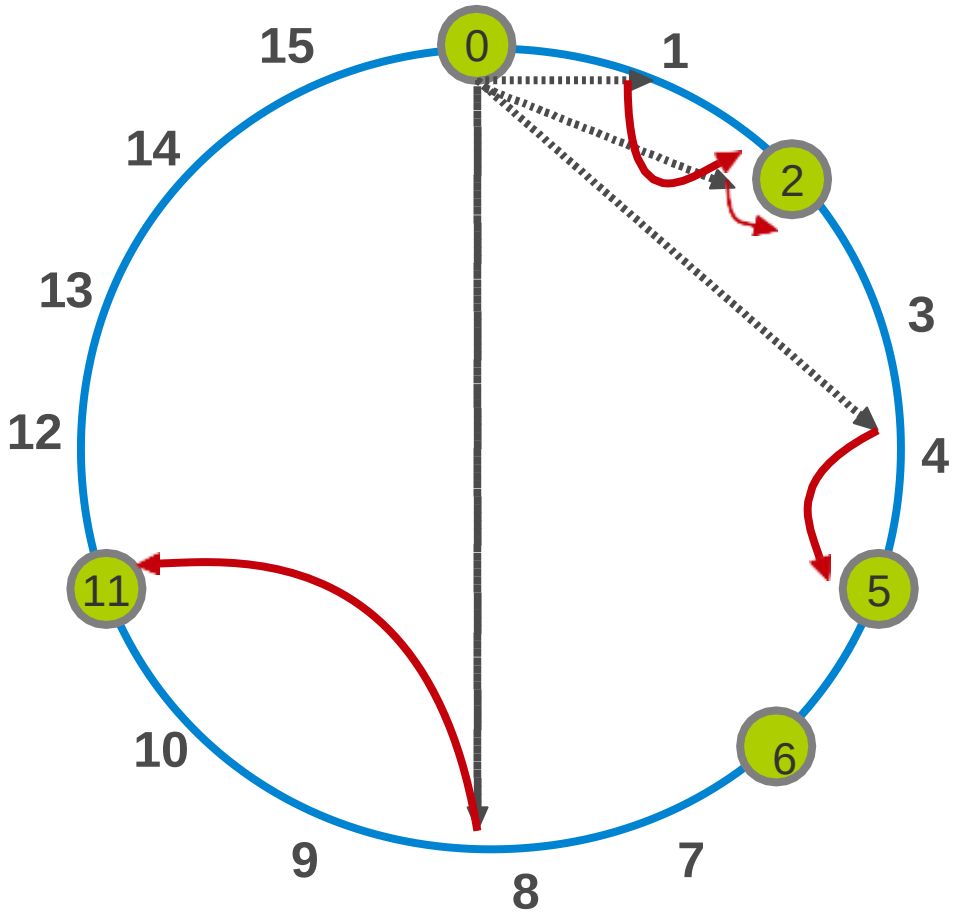


Buscas mais rápidas

- Fingertable

Aponta para succ(p+1)
Aponta para succ(p+2)
Aponta para succ(p+4)
Aponta para succ(p+8)
...
Aponta para succ(p+2^{M-1})

FingerTable nó 0	
i	Succ (n + 2^{i-1})
1	0 + 2^0 = 1 → 2
2	0 + 2^1 = 2 → 2
3	0 + 2^2 = 4 → 5
4	0 + 2^3 = 8 → 11



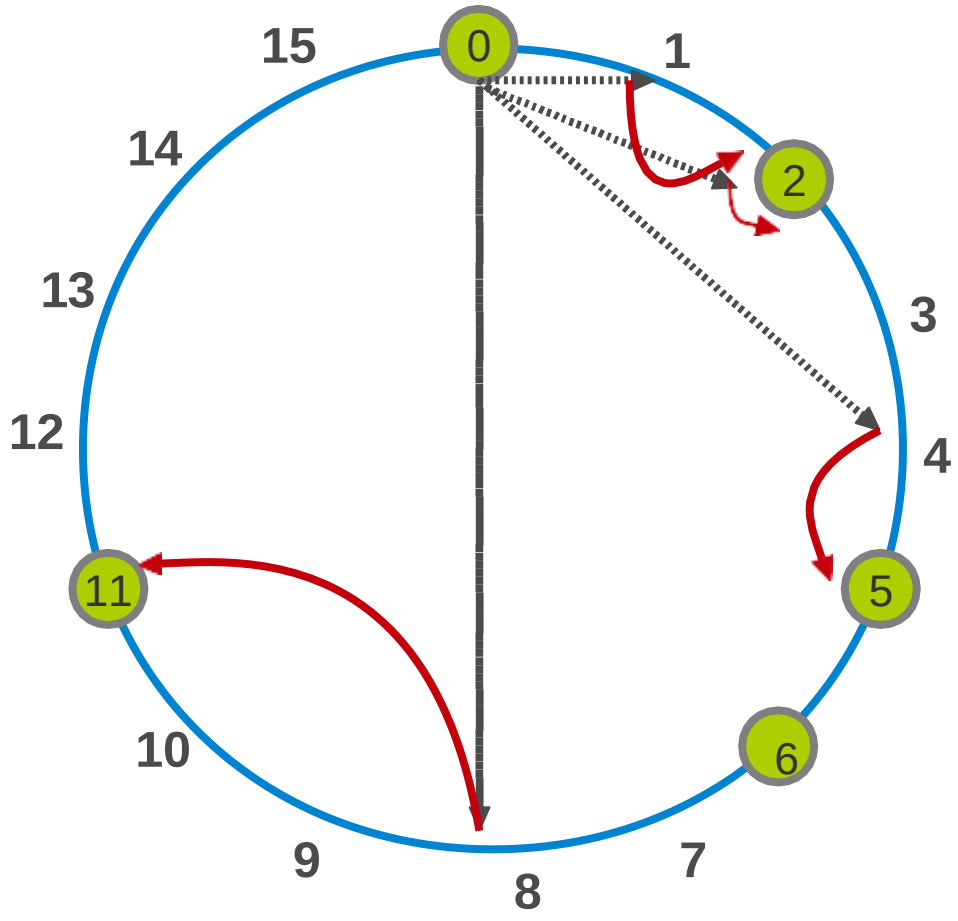
Buscas mais rápidas

- Tamanho da Fingertable é M e o total de chaves armazenadas será 2^M
- Cada nó p conhece o sucessor $(p + 2^{i-1})$ para i = 1 .. M
- $\log_2$ (N) saltos –hops– de um nó a qualquer outro para uma rede com N nós

Por que é importante o $\log_2$?
$\log_2(1000000) \sim 20$

Normalmente nas implementações
M = 128.

Na Figura:
M = 4, armazenando 16 chaves

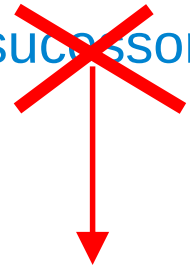


Buscas mais rápidas

```
// pergunta ao nó n encontrar o successor do id
procedure n.findSuccessor(id, No ini) {
    if (predecessor ≠ nil and id ∈ (predecessor, n]) then ini.response(id, n)
    else if (id ∈(n, successor]) then ini.response(id, sucessor)
    else // encaminha a pergunta pelo anel
        sucessor.findSucessor(id,ini)
}
```

Buscas mais rápidas

```
// pergunta ao nó n encontrar o successor do id
procedure n.findSuccessor(id, No ini) {
    if (predecessor ≠ nil and id ∈ (predecessor, n]) then ini.response(id, n)
    else if (id ∈(n, successor]) then ini.response(id, sucessor)
    else // encaminha a pergunta pelo anel
        sucessor.findSucessor(id,ini)
}
```



`closestPrecedingNode(id)`

Buscas mais rápidas

```
// pergunta ao nó n encontrar o successor do id
procedure n.findSuccessor(id, No ini) {
    if (predecessor ≠ nil and id ∈ (predecessor, n]) then ini.response(id, n)
    else if (id ∈(n, successor]) then ini.response(id, sucessor)
    else // encaminha a pergunta pelo anel
        m = closestPrecedingNode(id)
        m.findSucessor(id,ini)
}

// procura localmente pelo nó próximo ao id
procedure closestPrecedingNode(id) {
    for (i = M to 1 do) {
        if (FT[i] ∈(n, id)) then
            return FT[i]
    }
    return n // meu id
}
```

Algoritmos

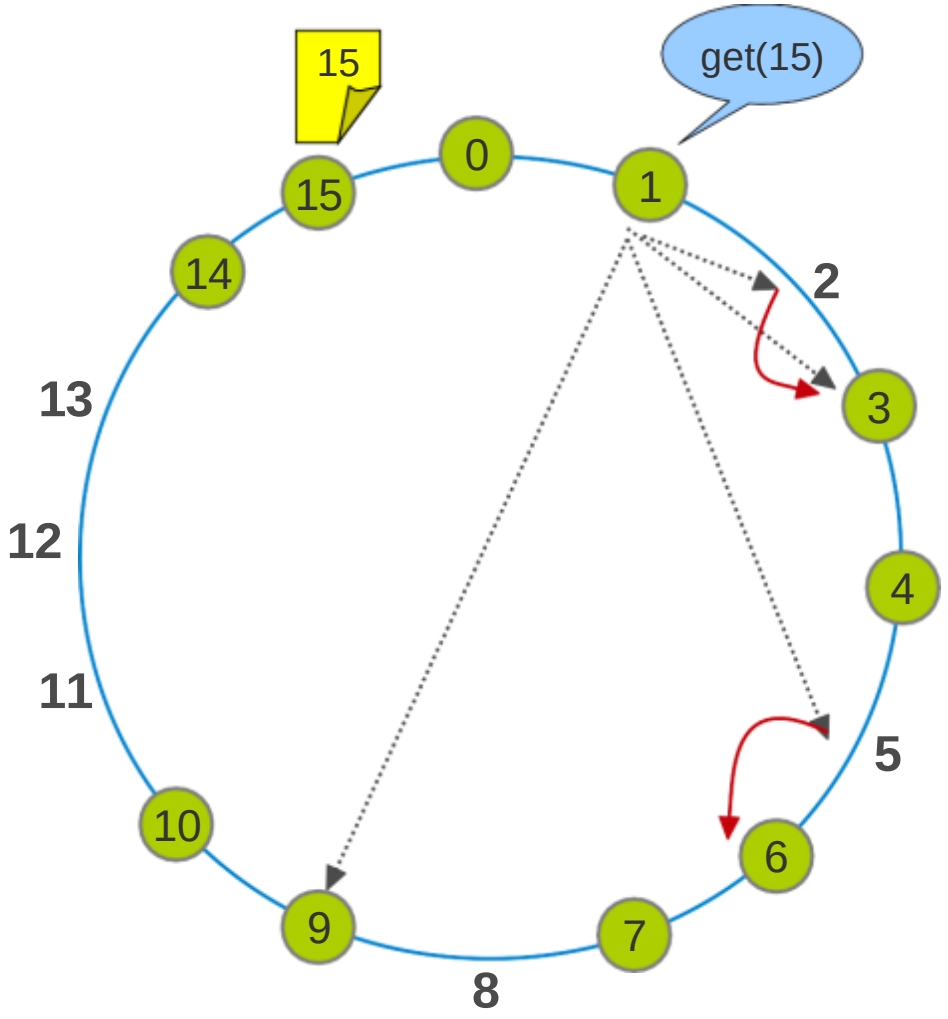
```
// Estrutura do nó n (considerando store)
No {
    int meuId = n
    No predecessor = null
    No sucessor = null
    KV chaves-valor = null
}

// Modificando a estrutura do nó n (agora considerando FT)
No {
    int meuId = n
    No predecessor = null
    No sucessor = null
    KV chaves-valor = null
    No[] FT = null
}
```

Buscas mais rápidas

```
procedure n.findSuccessor(id, No ini) {
    if (predecessor ≠ nil and id ∈ (predecessor, n]) then ini.response(id, n)
    else if (id ∈(n, successor]) then ini.response(id, sucessor)
    else // encaminha a pergunta pelo anel
        m = closestPrecedingNode(id)
        m.findSucessor(id,ini)
}
```

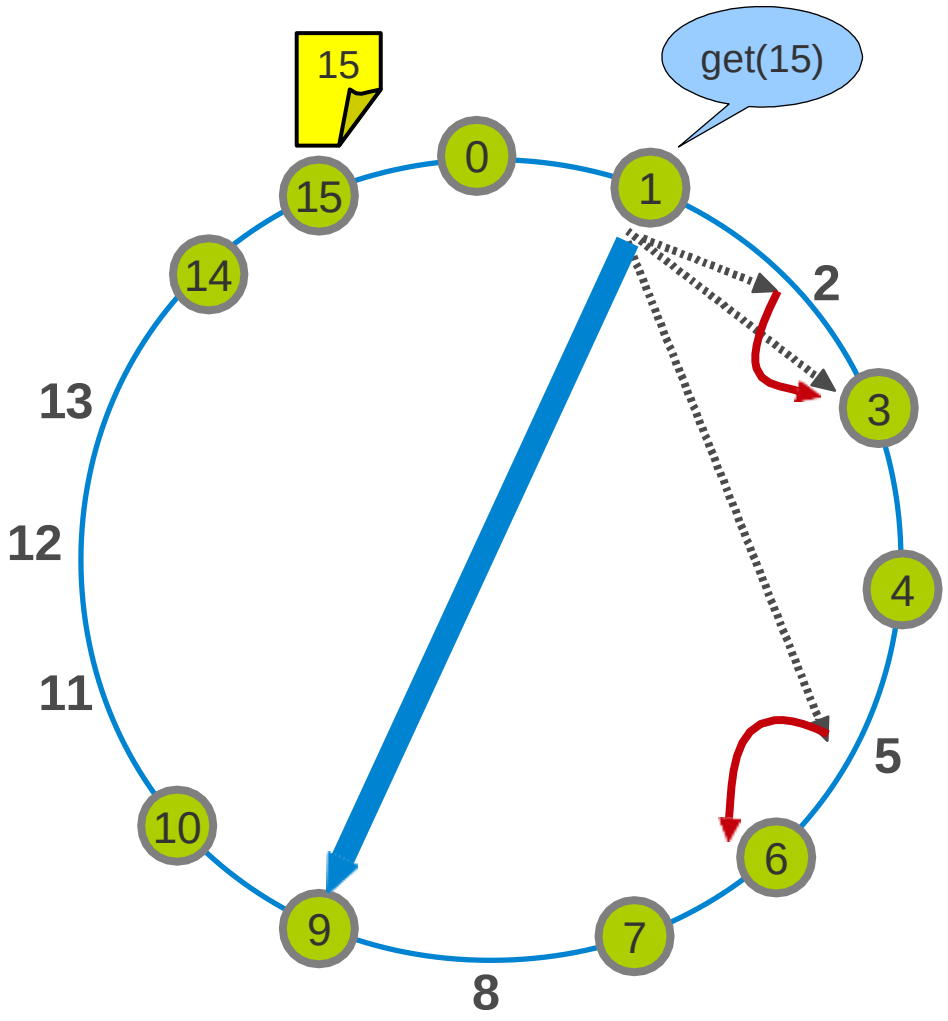
FingerTable nó 1	
i	Succ (n + 2^{i-1})
1	1 + 2^0 = 2 → 3
2	1 + 2^1 = 3 → 3
3	1 + 2^2 = 5 → 6
4	1 + 2^3 = 9 → 9



Buscas mais rápidas

```
procedure n.findSuccessor(id, No ini) {
    if (predecessor ≠ nil and id ∈ (predecessor, n]) then ini.response(id, n)
    else if (id ∈(n, successor]) then ini.response(id, sucessor)
    else // encaminha a pergunta pelo anel
        m = closestPrecedingNode(id)
        m.findSucessor(id,ini)
}
```

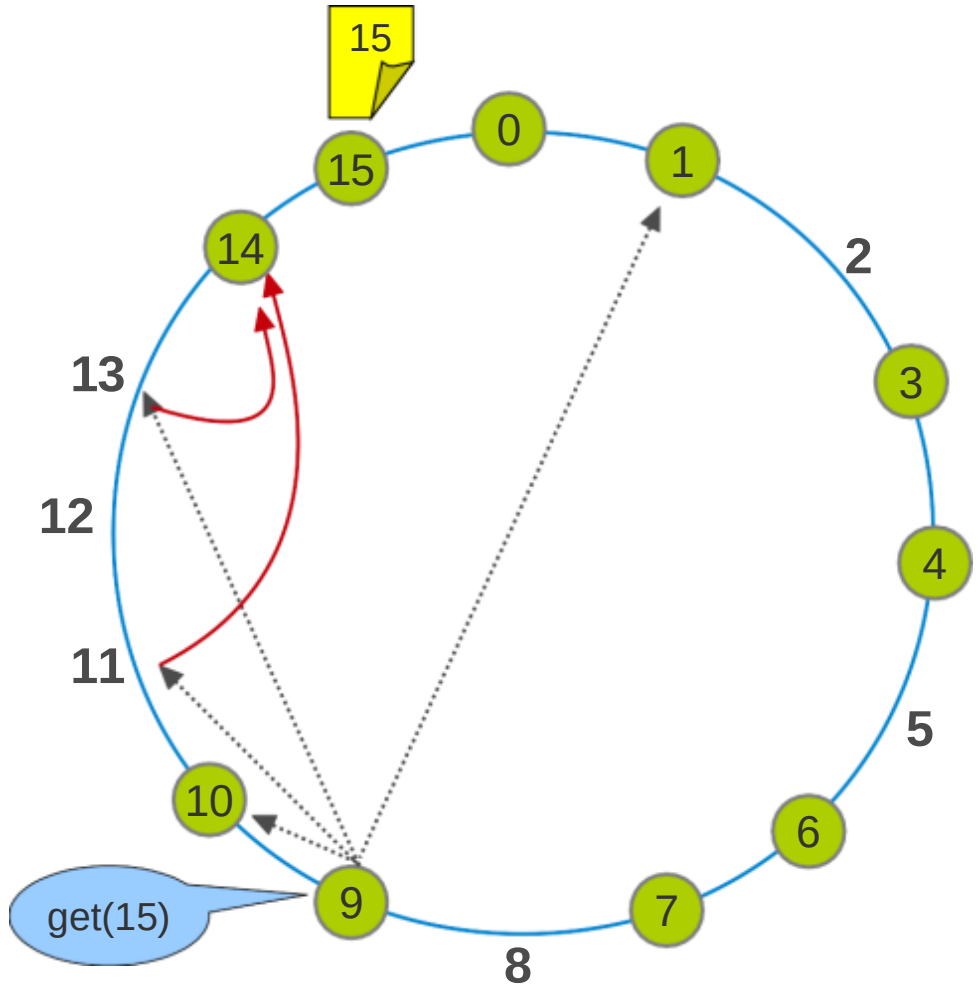
FingerTable nó 1	
i	Succ $(n + 2^{i-1})$
1	$1 + 2^0 = 2 \rightarrow 3$
2	$1 + 2^1 = 3 \rightarrow 3$
3	$1 + 2^2 = 5 \rightarrow 6$
4	$1 + 2^3 = 9 \rightarrow 9$



Buscas mais rápidas

```
procedure n.findSuccessor(id, No ini) {
    if (predecessor ≠ nil and id ∈ (predecessor, n]) then ini.response(id, n)
    else if (id ∈(n, successor]) then ini.response(id, sucessor)
    else // encaminha a pergunta pelo anel
        m = closestPrecedingNode(id)
        m.findSucessor(id,ini)
}
```

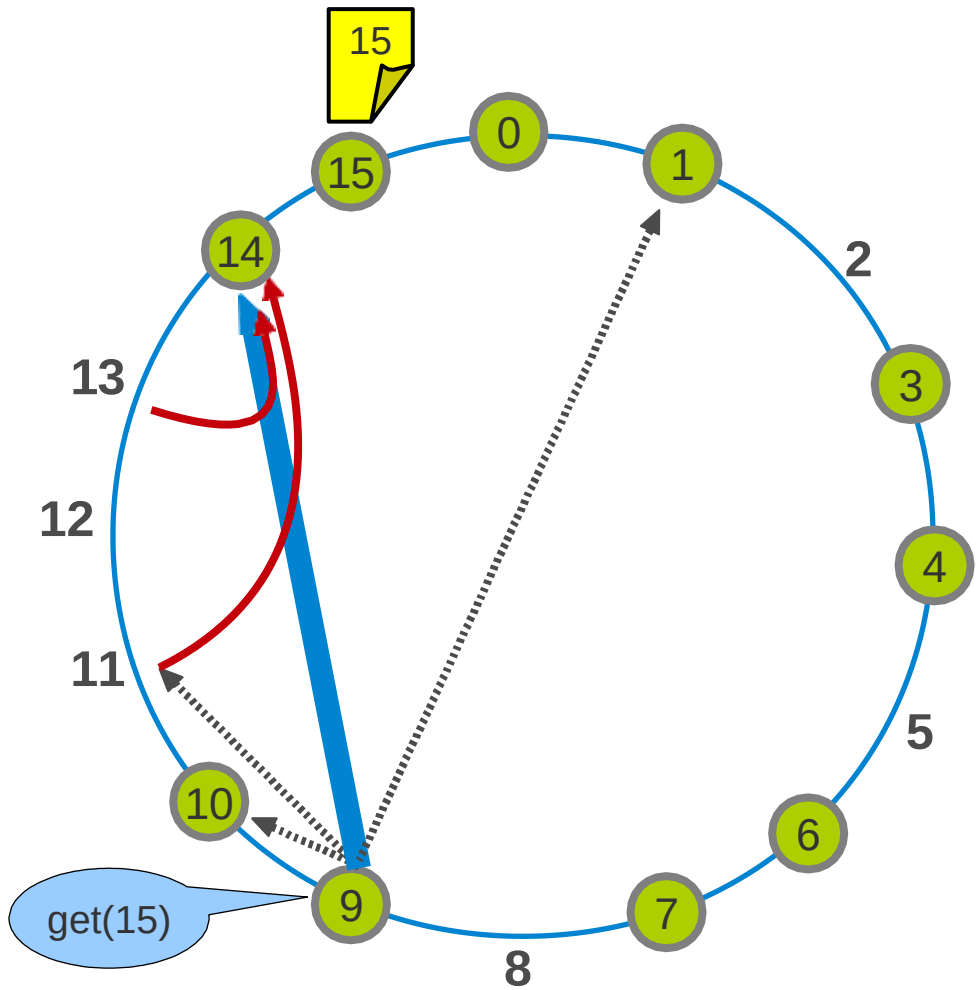
FingerTable nó 9	
i	Succ $(n + 2^{i-1})$
1	$9 + 2^0 = 10 \rightarrow 10$
2	$9 + 2^1 = 11 \rightarrow 14$
3	$9 + 2^2 = 13 \rightarrow 14$
4	$9 + 2^3 = 17 \% 16 \rightarrow 1$



Buscas mais rápidas

```
procedure n.findSuccessor(id, No ini) {
    if (predecessor ≠ nil and id ∈ (predecessor, n]) then ini.response(id, n)
    else if (id ∈(n, successor]) then ini.response(id, sucessor)
    else // encaminha a pergunta pelo anel
        m = closestPrecedingNode(id)
        m.findSucessor(id,ini)
}
```

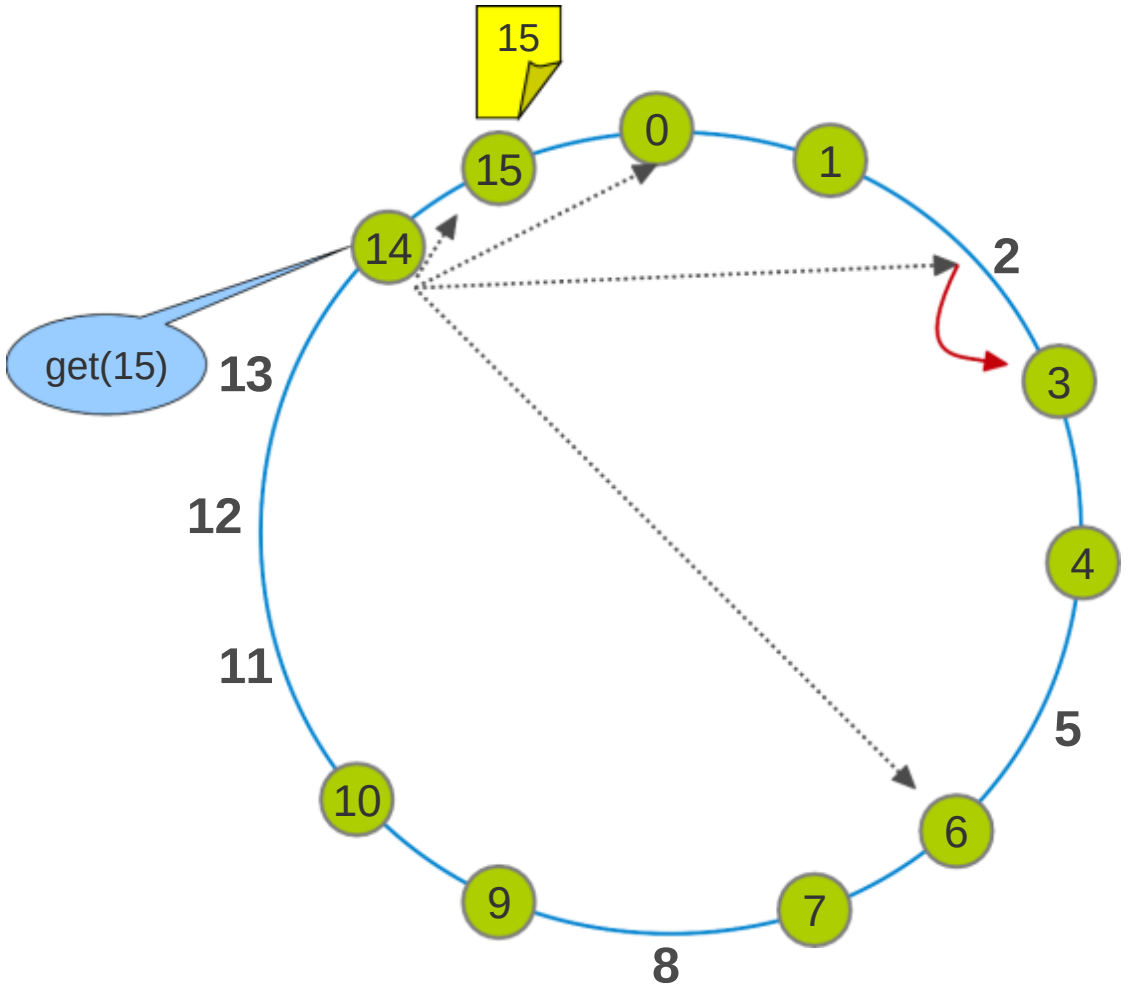
FingerTable nó 9	
i	Succ ($n + 2^{i-1}$)
1	$9 + 2^0 = 10 \rightarrow 10$
2	$9 + 2^1 = 11 \rightarrow 14$
3	$9 + 2^2 = 13 \rightarrow 14$
4	$9 + 2^3 = 17 \% 16 \rightarrow 1$



Buscas mais rápidas

```
procedure n.findSuccessor(id, No ini) {
    if (predecessor ≠ nil and id ∈ (predecessor, n]) then ini.response(id, n)
    else if (id ∈(n, successor]) then ini.response(id, sucessor)
    else // encaminha a pergunta pelo anel
        m = closestPrecedingNode(id)
        m.findSucessor(id,ini)
}
```

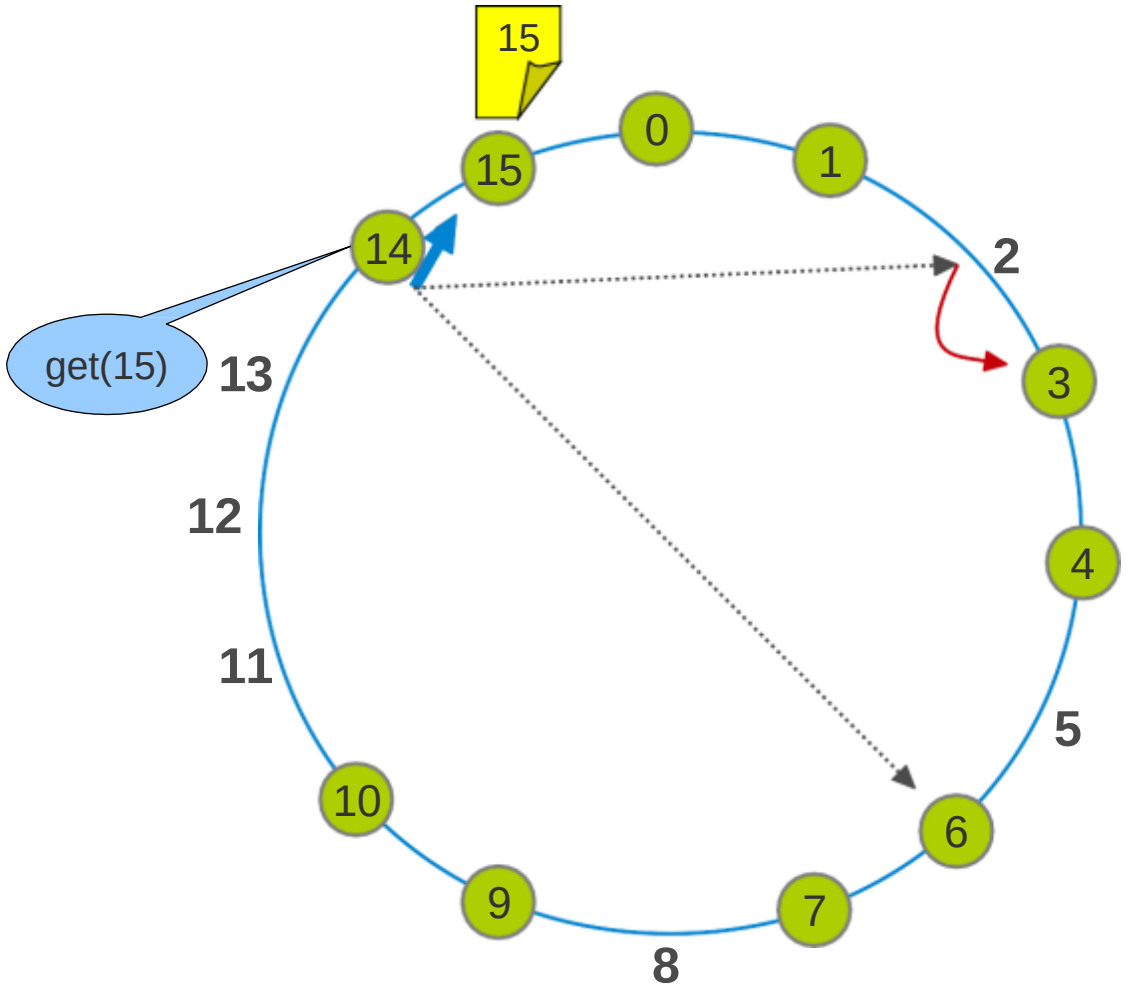
FingerTable nó 14	
I	Succ (n + 2^{i-1})
1	14 + 2^0 = 15 % 16 → 15
2	14 + 2^1 = 16 % 16 → 0
3	14 + 2^2 = 18 % 16 → 3
4	14 + 2^3 = 22 % 16 → 6



Buscas mais rápidas

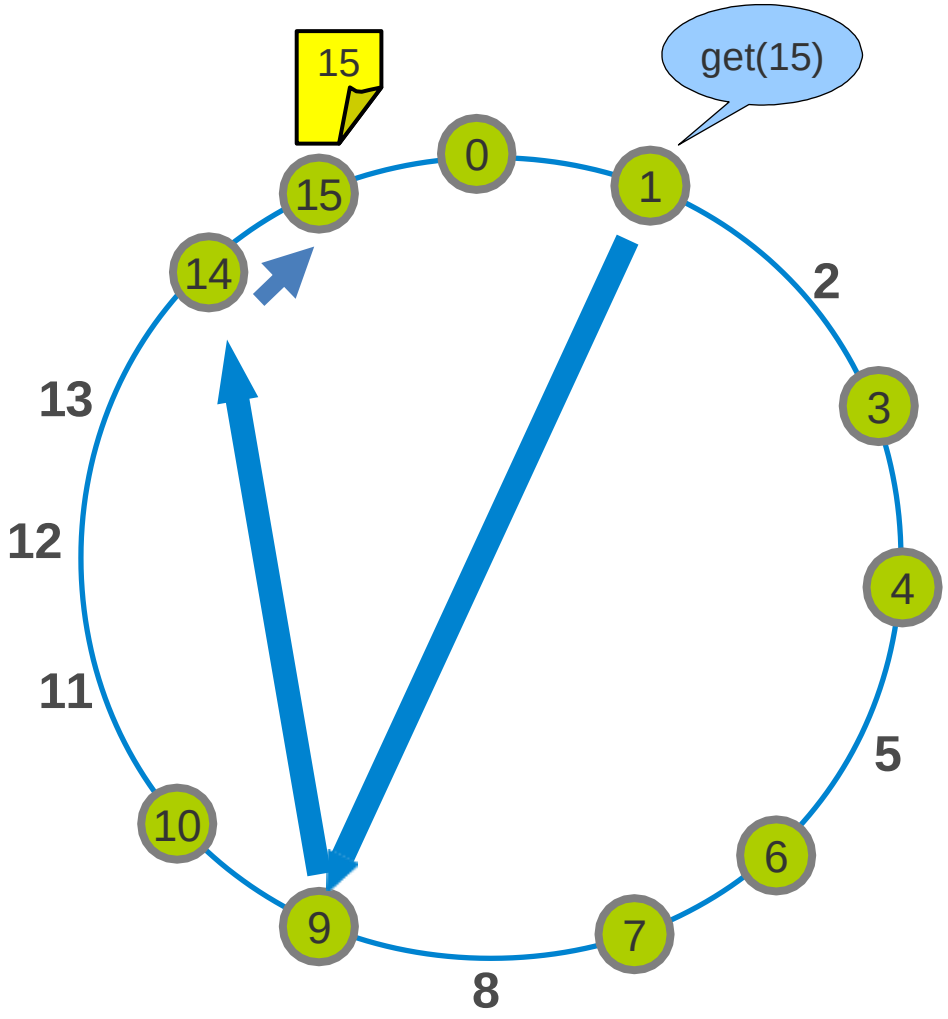
```
procedure n.findSuccessor(id, No ini) {
    if (predecessor ≠ nil and id ∈ (predecessor, n]) then ini.response(id, n)
    else if (id ∈(n, successor]) then ini.response(id, sucessor)
    else // encaminha a pergunta pelo anel
        m = closestPrecedingNode(id)
        m.findSucessor(id,ini)
}
```

FingerTable nó 14	
I	Succ (n + 2^{i-1})
1	14 + 2^0 = 15 % 16 → 15
2	14 + 2^1 = 16 % 16 → 0
3	14 + 2^2 = 18 % 16 → 3
4	14 + 2^3 = 22 % 16 → 6



Buscas mais rápidas

```
procedure n.findSuccessor(id, No ini) {
    if (predecessor ≠ nil and id ∈ (predecessor, n]) then ini.response(id, n)
    else if (id ∈(n, successor]) then ini.response(id, sucessor)
    else // encaminha a pergunta pelo anel
        m = closestPrecedingNode(id)
        m.findSucessor(id,ini)
}
```



Conceitos adquiridos

- Busca distribuída em O(log N)
- Fingertable
- Consistent Hashing.
- SPOF – Single Point of Failure.